

---

# ICPC Algorithm Library

---

# 目录

|                         |                                |                |           |                   |                  |
|-------------------------|--------------------------------|----------------|-----------|-------------------|------------------|
| <b>1 Basic</b>          | <b>1</b>                       |                |           |                   |                  |
| 1.1 test                | 1                              |                |           |                   |                  |
| 1.2 pai                 | 1                              |                |           |                   |                  |
| 1.3 缺省源                 | 1                              |                |           |                   |                  |
| 1.4 随机数                 | 1                              |                |           |                   |                  |
| 1.5 Mod                 | 1                              |                |           |                   |                  |
| 1.6 Comb                | 1                              |                |           |                   |                  |
| 1.7 PBDS Priority Queue | 1                              |                |           |                   |                  |
| <b>2 Graph</b>          | <b>2</b>                       |                |           |                   |                  |
| 2.1 差分约束                | 2                              |                |           |                   |                  |
| 2.2 欧拉回路                | 2                              |                |           |                   |                  |
| 2.3 Tarjan SCC          | 2                              |                |           |                   |                  |
| 2.4 Tarjan EBCC         | 2                              |                |           |                   |                  |
| 2.5 Tarjan VBCC         | 3                              |                |           |                   |                  |
| 2.6 最大流 Dinic           | 3                              |                |           |                   |                  |
| 2.7 费用流 EK              | 4                              |                |           |                   |                  |
| 2.8 上下界网络流              | 4                              |                |           |                   |                  |
| 2.9 三四元环                | 4                              |                |           |                   |                  |
| 2.10 支配树                | 4                              |                |           |                   |                  |
| 2.11 二分图最大匹配            | 5                              |                |           |                   |                  |
| 2.12 KM                 | 5                              |                |           |                   |                  |
| 2.13 2-SAT              | 6                              |                |           |                   |                  |
| 2.14 Segment Tree Graph | 6                              |                |           |                   |                  |
| <b>3 Data Structure</b> | <b>7</b>                       |                |           |                   |                  |
| 3.1 可撤销并查集              | 7                              |                |           |                   |                  |
| 3.2 单点修改线段树             | 7                              |                |           |                   |                  |
| 3.3 区间修改线段树             | 7                              |                |           |                   |                  |
| 3.4 线段树二分               | 8                              |                |           |                   |                  |
| 3.5 可持久化线段树/动态开点        | 8                              |                |           |                   |                  |
| 3.6 李超树                 | 9                              |                |           |                   |                  |
|                         |                                | 3.7 线段树合并/分裂   | 10        |                   | 6.9 万能欧几里得 19    |
|                         |                                | 3.8 ODT        | 10        |                   | 6.10 杜教筛 19      |
|                         |                                | 3.9 FHQ Treap  | 11        |                   | 6.11 Min_25 筛 19 |
|                         |                                | 3.10 pbds tree | 12        |                   |                  |
|                         |                                | 3.11 普通莫队      | 12        | <b>7 Geometry</b> | <b>20</b>        |
|                         |                                | 3.12 带修莫队      | 12        | 7.1 vector        | 20               |
|                         |                                | 3.13 回滚莫队      | 13        | 7.2 极角排序          | 20               |
|                         |                                | 3.14 莫队二次离线    | 13        | 7.3 seg_poly      | 21               |
|                         | <b>4 Tree</b>                  |                | <b>13</b> |                   |                  |
|                         | 4.1 LCA                        |                | 13        | <b>8 Misc</b>     | <b>21</b>        |
|                         | 4.2 点分治                        |                | 14        | 8.1 模拟退火          | 21               |
|                         | 4.3 虚树                         |                | 14        | 8.2 爬山法           | 21               |
|                         | 4.4 树 Hash                     |                | 14        | 8.3 常数表           | 22               |
|                         | <b>5 String</b>                |                | <b>14</b> |                   |                  |
|                         | 5.1 String Hash                |                | 14        |                   |                  |
|                         | 5.2 KMP                        |                | 14        |                   |                  |
|                         | 5.3 Z 函数                       |                | 15        |                   |                  |
|                         | 5.4 AC 自动机                     |                | 15        |                   |                  |
|                         | 5.5 SA                         |                | 15        |                   |                  |
|                         | 5.6 SAM                        |                | 15        |                   |                  |
|                         | 5.7 Manacher                   |                | 16        |                   |                  |
|                         | 5.8 PAM                        |                | 16        |                   |                  |
|                         | <b>6 Math</b>                  |                | <b>16</b> |                   |                  |
|                         | 6.1 ExGCD                      |                | 16        |                   |                  |
|                         | 6.2 BSGS / ExBSGS              |                | 17        |                   |                  |
|                         | 6.3 线性筛                        |                | 17        |                   |                  |
|                         | 6.4 Miller-Rabin / Pollard-Rho |                | 17        |                   |                  |
|                         | 6.5 Primitive Root             |                | 18        |                   |                  |
|                         | 6.6 exCRT                      |                | 18        |                   |                  |
|                         | 6.7 二次剩余                       |                | 18        |                   |                  |
|                         | 6.8 floor sum                  |                | 19        |                   |                  |

# 1 Basic

## 1.1 test

```
export fs="-fsanitize=address,undefined"
ulimit -s 1024000
ulimit -v 1024000
mk() { g++ -o $1 $1.cpp -O2; }
```

## 1.2 pai

```
pai() {
    mk a; mk bf; mk gen
    while true; do
        ./gen > 1.in
        ./bf < 1.in > 1.ans
        ./a < 1.in > 1.out
        if diff -Zq 1.out 1.ans; then echo ac; else echo wa; break; fi
    done
}
```

## 1.3 缺省源

```
#include <bits/stdc++.h>
using namespace std;

#define ALL(s) s.begin(), s.end()
#define SZ(s) int(s.size())
#define pb push_back

using LL = long long;
using ULL = unsigned long long;
using I = __int128;

void Cas() {
}

int main() {
    cin.tie(0)->sync_with_stdio(0);
    int t = 1;
    cin >> t;
    while (t--) {
        Cas();
    }
    return 0;
}
```

## 1.4 随机数

```
ULL seed = chrono::steady_clock::now().time_since_epoch().count();
mt19937_64 rnd(seed);
```

## 1.5 Mod

```
constexpr LL P = 998244353;
LL Add(LL x, LL y) { return (x += y) >= P ? x - P : x; }
LL Mul(LL x, LL y) { return x * y % P; }
LL qp(LL a, LL b = P - 2) {
    LL r = 1;
    for (; b; b >>= 1, a = Mul(a, a)) if (b & 1) r = Mul(r, a);
    return r;
}
```

## 1.6 Comb

```
vector<LL> fac, ifac;
void initComb(int n) {
    fac.resize(n + 1), ifac.resize(n + 1);
    fac[0] = 1;
    for (int i = 1; i <= n; i++) fac[i] = Mul(fac[i - 1], i);
    ifac[n] = qp(fac[n]);
    for (int i = n; i; i--) ifac[i - 1] = Mul(ifac[i], i);
}
LL binom(int n, int m) {
    if (n < m || m < 0) return 0;
    return Mul(fac[n], Mul(ifac[m], ifac[n - m]));
}
```

## 1.7 PBDS Priority Queue

- 当前写法为大根堆，top() 返回 x 最大的元素；改为 return x > b.x; 即为小根堆。
- auto it = h.push(x) 返回元素句柄；h.modify(it, y) 修改该元素，h.erase(it) 删除该元素。
- h.join(g) 将 g 破坏性合并到 h，之后 g 为空。

```
#include <bits/extc++.h>
using namespace __gnu_pbds;
struct V {
    int x;
    bool operator<(V b) const {
        return x < b.x;
    }
};
using Heap = __gnu_pbds::priority_queue<V, std::less<V>, pairing_heap_tag>;
```

## 2 Graph

### 2.1 差分约束

约束  $x_v - x_u \leq w$  连边  $u \rightarrow v$ , 边权为  $w$ 。

```
vector<LL> d(n);
vector<int> len(n), in(n, 1);
queue<int> q;
for (int u = 0; u < n; u++) q.push(u);
bool ok = 1;
while (SZ(q) && ok) {
    int u = q.front();
    q.pop(), in[u] = 0;
    for (auto [v, w] : G[u]) if (d[v] > d[u] + w) {
        d[v] = d[u] + w, len[v] = len[u] + 1;
        if (len[v] >= n) {
            ok = 0;
            break;
        }
        if (!in[v]) q.push(v), in[v] = 1;
    }
}
```

### 2.2 欧拉回路

// 有向图: 每条边在  $G$  中出现一次, 会清空  $G$ ; 答案需有  $m+1$  个点且首尾相同。

```
vector<int> euler(int s) {
    vector<int> st{s}, ans;
    while (SZ(st)) {
        int u = st.back();
        if (SZ(G[u])) {
            int v = G[u].back();
            G[u].pop_back(), st.pb(v);
        } else {
            ans.pb(u), st.pop_back();
        }
    }
    reverse(ALL(ans));
    return ans;
}
```

// 无向图:  $G[u]$  存  $\{v, id\}$ , 两端同  $id$ ; 答案需有  $m+1$  个点且首尾相同。

```
vector<int> euler(int s, int m) {
    vector<int> st{s}, ans;
    vector<char> used(m);
    while (SZ(st)) {
        int u = st.back();
        while (SZ(G[u]) && used[G[u].back().second]) G[u].pop_back();
        if (SZ(G[u])) {
```

```
            auto [v, id] = G[u].back();
            G[u].pop_back(), used[id] = 1, st.pb(v);
        } else {
            ans.pb(u), st.pop_back();
        }
    }
    reverse(ALL(ans));
    return ans;
}
```

### 2.3 Tarjan SCC

```
int tim = 0, scc = 0;
vector<int> dfn(n), low(n), bel(n, -1), stk;
auto tarj = [&](auto _, int u) -> void {
    dfn[u] = low[u] = ++tim;
    stk.push_back(u);
    for (int v : G[u]) {
        if (!dfn[v]) {
            _(_, v);
            low[u] = min(low[u], low[v]);
        } else if (bel[v] == -1) {
            low[u] = min(low[u], dfn[v]);
        }
    }
    if (dfn[u] != low[u]) return;
    while (1) {
        int v = stk.back();
        stk.pop_back();
        bel[v] = scc;
        if (v == u) break;
    }
    scc++;
};
for (int u = 0; u < n; u++) {
    if (!dfn[u]) tarj(tarj, u);
}
```

### 2.4 Tarjan EBCC

```
//  $G[u]$  存  $\{v, id\}$ , 无向边  $id$  为  $0 \dots m-1$ 。
int tim = 0, ebcc = 0;
vector<int> dfn(n), low(n), bel(n, -1);
vector<char> bridge(m);
auto tarj = [&](auto self, int u, int pe) -> void {
    dfn[u] = low[u] = ++tim;
    for (auto [v, id] : G[u]) {
        if (!dfn[v]) {
            self(self, v, id);
```

```

    low[u] = min(low[u], low[v]);
    if (low[v] > dfn[u]) bridge[id] = 1;
} else if (id != pe) {
    low[u] = min(low[u], dfn[v]);
}
}
};
for (int u = 0; u < n; u++) if (!dfn[u]) tarj(tarj, u, -1);
auto paint = [&](auto self, int u) -> void {
    bel[u] = ebcc;
    for (auto [v, id] : G[u])
        if (bel[v] == -1 && !bridge[id]) self(self, v);
};
for (int u = 0; u < n; u++) if (bel[u] == -1) paint(paint, u), ebcc++;

```

## 2.5 Tarjan VBCC

```

// G[u] 存 {v,id}, bcc 为点双点集, cut 标记割点。
int tim = 0;
vector<int> dfn(n), low(n), stk;
vector<char> cut(n);
vector<vector<int>> bcc;
auto tarj = [&](auto self, int u, int pe) -> void {
    dfn[u] = low[u] = ++tim;
    stk.pb(u);
    int son = 0;
    for (auto [v, id] : G[u]) {
        if (id == pe) continue;
        if (!dfn[v]) {
            son++, self(self, v, id);
            low[u] = min(low[u], low[v]);
            if (low[v] >= dfn[u]) {
                if (pe != -1 || son > 1) cut[u] = 1;
                bcc.pb({u});
                while (bcc.back().back() != v) {
                    bcc.back().pb(stk.back());
                    stk.pop_back();
                }
            }
        } else {
            low[u] = min(low[u], dfn[v]);
        }
    }
};
for (int u = 0; u < n; u++) if (!dfn[u]) {
    if (!SZ(G[u])) dfn[u] = low[u] = ++tim, bcc.pb({u});
    else tarj(tarj, u, -1), stk.clear();
}

```

## 2.6 最大流 Dinic

```

struct Dinic {
    struct E {
        int v;
        LL w;
    };
    static constexpr LL inf = LLONG_MAX / 4;
    vector<E> e;
    vector<vector<int>> G;
    vector<int> d, cur;
    Dinic(int n) : G(n), d(n), cur(n) {}
    void add(int u, int v, LL w) {
        G[u].pb(SZ(e)), e.pb({v, w});
        G[v].pb(SZ(e)), e.pb({u, 0});
    }
    bool bfs(int s, int t) {
        fill(ALL(d), -1), d[s] = 0;
        queue<int> q;
        q.push(s);
        while (SZ(q)) {
            int u = q.front();
            q.pop();
            for (int id : G[u]) {
                auto [v, w] = e[id];
                if (w && d[v] == -1) d[v] = d[u] + 1, q.push(v);
            }
        }
        return d[t] != -1;
    }
    LL dfs(int u, int t, LL f) {
        if (u == t) return f;
        for (int &i = cur[u]; i < SZ(G[u]); i++) {
            int id = G[u][i];
            auto &[v, w] = e[id];
            if (w && d[v] == d[u] + 1) {
                LL x = dfs(v, t, min(f, w));
                if (x) {
                    w -= x, e[id ^ 1].w += x;
                    return x;
                }
            }
        }
        return 0;
    }
    LL flow(int s, int t) {
        LL ans = 0, f;
        while (bfs(s, t)) {
            fill(ALL(cur), 0);
            while ((f = dfs(s, t, inf))) ans += f;
        }
    }
};

```

```

    }
    return ans;
}
};

```

## 2.7 费用流 EK

// 残量网络无负环时使用。

```

struct MCMF {
    struct E {
        int u, v;
        LL w, c;
    };
    static constexpr LL inf = LLONG_MAX / 4;
    vector<E> e;
    vector<vector<int>>> G;
    vector<LL> d, in;
    vector<int> p, vis;
    MCMF(int n) : G(n), d(n), in(n), p(n), vis(n) {}
    void add(int u, int v, LL w, LL c) {
        G[u].pb(SZ(e)), e.pb({u, v, w, c});
        G[v].pb(SZ(e)), e.pb({v, u, 0, -c});
    }
    bool spfa(int s, int t) {
        fill(ALL(d), inf), fill(ALL(vis), 0), d[s] = 0, in[s] = inf;
        queue<int> q;
        q.push(s), vis[s] = 1;
        while (SZ(q)) {
            int u = q.front();
            q.pop(), vis[u] = 0;
            for (int id : G[u]) {
                auto &x = e[id];
                if (x.w && d[x.v] > d[u] + x.c) {
                    d[x.v] = d[u] + x.c, p[x.v] = id;
                    in[x.v] = min(in[u], x.w);
                    if (!vis[x.v]) q.push(x.v), vis[x.v] = 1;
                }
            }
        }
        return d[t] != inf;
    }
    pair<LL, LL> flow(int s, int t, LL lim = inf) {
        LL f = 0, c = 0;
        while (f < lim && spfa(s, t)) {
            LL x = min(in[t], lim - f);
            f += x, c += x * d[t];
            for (int v = t; v != s; v = e[p[v]].u) {
                int id = p[v];
                e[id].w -= x, e[id ^ 1].w += x;
            }
        }
    }
};

```

```

    }
    return {f, c};
}
};

```

## 2.8 上下界网络流

对原边  $u \rightarrow v$ 、下界  $l$ 、上界  $r$ ，统一加入容量为  $r-l$  的边，并令  $b_u -= l, b_v += l$ 。再加入超级源汇  $SS, TT$ ： $b_u > 0$  时连  $SS \rightarrow u$ 、容量  $b_u$ ； $b_u < 0$  时连  $u \rightarrow TT$ 、容量  $-b_u$ 。

- 无源汇可行流：跑  $SS \rightarrow TT$  最大流，所有  $SS$  出边满流当且仅当有解；原边流量为下界加残量网络中的流量。
- 有源汇  $s, t$  可行流：额外加入  $t \rightarrow s$ 、容量  $\infty$ ，再按无源汇可行流处理。
- 最大流：记可行流中  $t \rightarrow s$  的流量为  $F$ ，删去  $SS, TT$  及  $t \rightarrow s$  后跑  $s \rightarrow t$ ，答案为  $F$  加新增流量。
- 最小流：从同一份可行残量网络出发，删去辅助点边后跑  $t \rightarrow s$ ，答案为  $F$  减新增流量。
- 最小费用可行流：保留下界费用  $\sum lc$ ，在上述建图上求满流的最小费用，再消残量网络负环。

## 2.9 三四元环

```

// 简单无向图，O(m sqrt(m)) 统计三元环 c3、四元环 c4。
auto lt = [&](int u, int v) {
    return pair(SZ(G[u]), u) < pair(SZ(G[v]), v);
};
vector<vector<int>>> D(n);
for (int u = 0; u < n; u++)
    for (int v : G[u]) if (lt(u, v)) D[u].pb(v);
LL c3 = 0, c4 = 0;
vector<int> vis(n, -1), cnt(n);
for (int u = 0; u < n; u++) {
    for (int v : D[u]) vis[v] = u;
    for (int v : D[u])
        for (int w : D[v]) if (vis[w] == u) c3++;
    for (int v : D[u])
        for (int w : G[v]) if (lt(u, w)) c4 += cnt[w]++;
    for (int v : D[u])
        for (int w : G[v]) if (lt(u, w)) cnt[w] = 0;
}

```

## 2.10 支配树

在有向图中固定起点  $s$ 。若从  $s$  到  $v$  的每条路径都经过  $u$ ，则称  $u$  支配  $v$ ； $v$  最近的严格支配点记为  $idom(v)$ 。所有  $idom(v) \rightarrow v$  构成支配树，删去  $u$  后无法再由  $s$  到

达的点，正是  $u$  在支配树子树中的点。常用于控制流图 and “必经点” 问题。

Lengauer–Tarjan 的实现流程：

1. 从  $s$  DFS，只给可达点编号；rk 是 DFS 序到原编号的映射，fa 是 DFS 树父亲，rg 保存反图。
2. 按 DFS 序倒序求半支配点 sd。并查集 uf 上的 get(v) 返回路径中 sd 最小的点，mn 记录这个最优点。
3. 将点放入 b[sd]；把当前点连向 DFS 树父亲后，处理父亲的桶，得到临时直接支配点 id。
4. 按 DFS 序正序修正 id，再映射回原编号。dom( $G, s$ ) 返回每点的直接支配点；根返回自身，不可达点返回  $-1$ 。

时间复杂度  $O((n+m)\log n)$ ，其中  $n, m$  是从  $s$  可达部分的点数和边数。

```
vector<int> dom(vector<vector<int>> &G, int s) {
    int n = SZ(G), t = 0;
    vector<int> dfn(n, -1), rk(n), fa(n, -1);
    auto dfs = [&](auto self, int u) -> void {
        dfn[u] = t, rk[t++] = u;
        for (int v : G[u]) if (dfn[v] == -1) {
            self(self, v), fa[dfn[v]] = dfn[u];
        }
    };
    dfs(dfs, s);
    vector<vector<int>> rg(t), b(t);
    for (int u = 0; u < n; u++) if (dfn[u] != -1)
        for (int v : G[u]) rg[dfn[v]].pb(dfn[u]);
    vector<int> sd(t), id(t, -1), uf(t, -1), mn(t);
    iota(ALL(sd), 0), iota(ALL(mn), 0);
    auto up = [&](auto self, int x) -> void {
        int y = uf[x];
        if (y != -1 && uf[y] != -1) {
            self(self, y);
            if (sd[mn[y]] < sd[mn[x]]) mn[x] = mn[y];
            uf[x] = uf[y];
        }
    };
    auto get = [&](int x) {
        if (uf[x] == -1) return mn[x];
        up(up, x);
        int y = uf[x];
        return sd[mn[y]] < sd[mn[x]] ? mn[y] : mn[x];
    };
    for (int w = t - 1; w; w--) {
        for (int v : rg[w]) sd[w] = min(sd[w], sd[get(v)]);
        b[sd[w]].pb(w), uf[w] = fa[w];
        for (int v : b[fa[w]]) {
            int u = get(v);
            id[v] = sd[u] < sd[v] ? u : fa[w];
        }
    }
}
```

```
    }
    b[fa[w]].clear();
}
for (int u = 1; u < t; u++) if (id[u] != sd[u]) id[u] = id[id[u]];
vector<int> res(n, -1);
res[s] = s;
for (int u = 1; u < t; u++) res[rk[u]] = rk[id[u]];
return res;
}
```

## 2.11 二分图最大匹配

设最大匹配大小为  $\nu$ 。König 定理给出最小点覆盖大小为  $\nu$ ，最大独立集大小为  $|L| + |R| - \nu$ ；无孤立点时最小边覆盖大小为  $|L| + |R| - \nu$ 。DAG 的顶点不相交最小路径覆盖可拆点建二分图，答案为  $|V| - \nu$ 。

求出最大匹配后，从所有未匹配左点沿“左到右非匹配边、右到左匹配边”搜索，记可达点集为  $Z$ 。最小点覆盖为  $(L \setminus Z) \cup (R \cap Z)$ ，其补集即最大独立集。覆盖左部的匹配存在当且仅当 Hall 条件成立；若  $|L| = |R|$ ，它就是完美匹配。

```
// 左部 0...n-1, 右部 0...m-1, G 为左到右的边。
vector<int> mt(m, -1), vis(n);
auto aug = [&](auto self, int u) -> bool {
    if (vis[u]) return 0;
    vis[u] = 1;
    for (int v : G[u]) {
        if (mt[v] == -1 || self(self, mt[v])) {
            mt[v] = u;
            return 1;
        }
    }
    return 0;
};
int matching = 0;
for (int u = 0; u < n; u++) {
    fill(ALL(vis), 0);
    matching += aug(aug, u);
}
```

## 2.12 KM

```
// n*n 完备二分图最大权完美匹配；补 0 点可求非完美匹配。
LL km(vector<vector<LL>> a) {
    int n = SZ(a);
    vector<LL> lx(n), ly(n), slack(n);
    vector<int> mt(n, -1);
    vector<char> vx(n), vy(n);
    for (int u = 0; u < n; u++) lx[u] = *max_element(ALL(a[u]));
    auto aug = [&](auto self, int u) -> bool {

```

```

if (vx[u]) return 0;
vx[u] = 1;
for (int v = 0; v < n; v++) if (!vy[v]) {
    LL d = lx[u] + ly[v] - a[u][v];
    if (d) {
        slack[v] = min(slack[v], d);
    } else {
        vy[v] = 1;
        if (mt[v] == -1 || self(self, mt[v])) {
            mt[v] = u;
            return 1;
        }
    }
}
return 0;
};
for (int s = 0; s < n; s++) {
    fill(ALL(slack), LLONG_MAX / 4);
    while (1) {
        fill(ALL(vx), 0), fill(ALL(vy), 0);
        if (aug(aug, s)) break;
        LL d = LLONG_MAX / 4;
        for (int v = 0; v < n; v++) if (!vy[v]) d = min(d, slack[v]);
        for (int u = 0; u < n; u++) if (vx[u]) lx[u] -= d;
        for (int v = 0; v < n; v++) {
            if (vy[v]) ly[v] += d;
            else slack[v] -= d;
        }
    }
}
LL ans = 0;
for (int v = 0; v < n; v++) ans += a[mt[v]][v];
return ans;
}

```

## 2.13 2-SAT

- $u_i = 2 \times u + i$
- $u_a \vee v_b$ : 加入  $u_{a \oplus 1} \rightarrow v_b$  和  $v_{b \oplus 1} \rightarrow u_a$
- $u_a \Rightarrow v_b$ : 加入  $u_a \rightarrow v_b$  和  $v_{b \oplus 1} \rightarrow u_{a \oplus 1}$
- 强制  $u = a$ : 加入  $u_{a \oplus 1} \rightarrow u_a$
- 禁止  $u = a$  与  $v = b$  同时成立: 加入  $u_a \rightarrow v_{b \oplus 1}$  和  $v_b \rightarrow u_{a \oplus 1}$
- $u = v$ : 加入  $u_0 \leftrightarrow v_0$  和  $u_1 \leftrightarrow v_1$
- $u \neq v$ : 加入  $u_0 \leftrightarrow v_1$  和  $u_1 \leftrightarrow v_0$

其中每个  $\leftrightarrow$  都表示正反两条有向边。

若存在  $u$  满足  $u_0, u_1$  在同一强连通分量中, 则无解。

构造:  $\text{ans}_u = [\text{bel}_{u_0} > \text{bel}_{u_1}]$ 。

## 2.14 Segment Tree Graph

- 原图和区间均使用 0-index: 原图节点编号为  $0 \sim n-1$ ,  $\text{mdf}(l, r, \dots)$  中的  $[l, r]$  为闭区间。
- ZKW 线段树内部仍使用从 1 开始的堆式编号, 叶子位置为  $S \sim 2S-1$ , 原图节点  $i$  对应叶子  $S+i$ 。
- $\text{sn}$  的值为  $2S-1$ , 表示一棵完整 ZKW 线段树的节点数。
- 对于  $1 \leq p \leq \text{sn}$ ,  $\text{id}(p, 0)$  为  $n+p-1$ , 表示 down 树节点;  $\text{id}(p, 1)$  为  $n+\text{sn}+p-1$ , 表示 up 树节点。
- $\text{tot}$  初始为  $n+2(2S-1)$ , 表示当前节点数, 也是下一个可用节点编号; 初始有效编号为  $0 \sim \text{tot}-1$ ,  $\text{Node}()$  返回  $\text{tot}$  后将其加一。

```

int S = 1;
while (S < n) S <<= 1;
int sn = 2 * S - 1;
int tot = n + 2 * sn;
vector<vector<pair<int, LL>>> G(tot);
auto id = [&](int p, int up) {
    return n + p - 1 + up * sn;
};
auto add = [&](int u, int v, LL w) {
    G[u].push_back({v, w});
};
auto link = [&](int u, int v, LL w, int up) {
    if (up) swap(u, v);
    add(u, v, w);
};
auto Node = [&]() {
    G.emplace_back();
    return tot++;
};
for (int up = 0; up < 2; up++) {
    for (int p = 1; p < S; p++) {
        link(id(p, up), id(p * 2, up), 0, up);
        link(id(p, up), id(p * 2 + 1, up), 0, up);
    }
    for (int i = 0; i < n; i++) {
        link(id(S + i, up), i, 0, up);
    }
}
auto mdf = [&](int l, int r, int x, LL w, int up) {
    for (l += S, r += S + 1; l < r; l >>= 1, r >>= 1) {
        if (l & 1) link(x, id(l++, up), w, up);
        if (r & 1) link(x, id(--r, up), w, up);
    }
};

```

```
// x -> [l, r]
mdf(l, r, x, w, 0);
// [l, r] -> x
mdf(l, r, x, w, 1);
// [l1, r1] -> [l2, r2]
int x = Node();
mdf(l1, r1, x, 0, 1);
mdf(l2, r2, x, w, 0);
```

## 3 Data Structure

### 3.1 可撤销并查集

不使用路径压缩，按大小合并；find、merge 单次  $O(\log n)$ 。用 `t=dsu.time()` 保存状态，`dsu.rollback(t)` 撤销此后的所有成功合并。点为  $\theta$ -index；`init(n)` 可初始化并用于多测清空。

```
struct DSU {
    vector<int> f, sz;
    vector<array<int, 2>> his;
    DSU(int n = 0) { init(n); }
    void init(int n) {
        f.resize(n), iota(ALL(f), 0);
        sz.assign(n, 1), his.clear();
    }
    int find(int x) {
        while (x != f[x]) x = f[x];
        return x;
    }
    bool same(int x, int y) {
        return find(x) == find(y);
    }
    bool merge(int x, int y) {
        x = find(x), y = find(y);
        if (x == y) return false;
        if (sz[x] < sz[y]) swap(x, y);
        his.push_back({x, y});
        sz[x] += sz[y], f[y] = x;
        return true;
    }
    int size(int x) {
        return sz[find(x)];
    }
    int time() {
        return SZ(his);
    }
    void rollback(int t) {
        while (SZ(his) > t) {
```

```
        auto [x, y] = his.back();
        his.pop_back();
        f[y] = y, sz[x] -= sz[y];
    }
};
```

### 3.2 单点修改线段树

- $\theta$ -index，左闭右开  $[l, r)$ 。
- 本模板不将  $n$  补齐到 2 的幂，不能直接读取 `t[1]` 查询全局结果，须使用 `prod(0,  $\hookrightarrow$  n)`。

```
template <class S> struct SGT {
    int n;
    vector<S> t;
    SGT(int n) : n(n), t(2 * n) {}
    SGT(vector<S> a) : SGT(SZ(a)) {
        copy(ALL(a), t.begin() + n);
        for (int p = n - 1; p; p--) t[p] = t[2 * p] + t[2 * p + 1];
    }
    void set(int p, S x) {
        t[p += n] = x;
        while (p >= 1) t[p] = t[2 * p] + t[2 * p + 1];
    }
    S prod(int l, int r) {
        S x{}, y{};
        for (l += n, r += n; l < r; l >= 1, r >= 1) {
            if (l & 1) x = x + t[l++];
            if (r & 1) y = t[--r] + y;
        }
        return x + y;
    }
};
```

```
struct S {
    LL mx = -INF;
};
S operator+(S a, S b) {
    return {max(a.mx, b.mx)};
}
```

### 3.3 区间修改线段树

- $\theta$ -index，左闭右开  $[l, r)$ 。
- 按题目补全 `V`、`L::operator=`、`merge` 和 `apply`，其中 `L{}` 为空标记。
- 多组测试每组先用 `seg={}`；清空整棵线段树，再重新 `build`。

```
#define ls (p << 1)
```

```

#define rs (p << 1 | 1)
struct SGT {
    struct V {
    };
    struct L {
        bool operator==(L b) {}
    };
    int n;
    V t[N << 2];
    L lz[N << 2];
    V merge(V a, V b) {}
    void apply(int p, int l, int r, L x) {}
    void up(int p) {
        t[p] = merge(t[l], t[r]);
    }
    void down(int p, int l, int r) {
        if (lz[p] == L{}) return;
        int m = (l + r) / 2;
        apply(ls, l, m, lz[p]);
        apply(rs, m, r, lz[p]);
        lz[p] = {};
    }
    void build(const vector<V>& a, int p = 1, int l = 0, int r = -1) {
        if (r < 0) n = r = SZ(a);
        if (r - l == 1) {
            t[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(a, ls, l, m);
        build(a, rs, m, r);
        up(p);
    }
    void mdf(int ql, int qr, L x, int p = 1, int l = 0, int r = -1) {
        if (r < 0) r = n;
        if (ql <= l && r <= qr) return apply(p, l, r, x);
        down(p, l, r);
        int m = (l + r) / 2;
        if (ql < m) mdf(ql, qr, x, ls, l, m);
        if (m < qr) mdf(ql, qr, x, rs, m, r);
        up(p);
    }
    V qry(int ql, int qr, int p = 1, int l = 0, int r = -1) {
        if (r < 0) r = n;
        if (ql <= l && r <= qr) return t[p];
        down(p, l, r);
        int m = (l + r) / 2;
        if (qr <= m) return qry(ql, qr, ls, l, m);
        if (m <= ql) return qry(ql, qr, rs, m, r);
        return merge(qry(ql, qr, ls, l, m), qry(ql, qr, rs, m, r));
    }
};

```

```

};
#undef ls
#undef rs

SGT seg;
seg = {};
seg.build(a);
seg.mdf(l, r, x);
auto ans = seg.qry(l, r);

```

### 3.4 线段树二分

- 将下列函数直接插入上一节 SGT;  $\theta$ -index, 左闭右开  $[l, r)$ , 无解返回  $-1$ 。
- 按题目补全空的 `if`, 条件表示当前节点区间内没有答案。

```

int find_l(int ql, int qr, int p = 1, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (r <= ql || qr <= l) return -1;
    if () return -1;
    if (r - l == 1) return l;
    down(p, l, r);
    int m = (l + r) / 2;
    int q = find_l(ql, qr, ls, l, m);
    return q < 0 ? find_l(ql, qr, rs, m, r) : q;
}

int find_r(int ql, int qr, int p = 1, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (r <= ql || qr <= l) return -1;
    if () return -1;
    if (r - l == 1) return l;
    down(p, l, r);
    int m = (l + r) / 2;
    int q = find_r(ql, qr, rs, m, r);
    return q < 0 ? find_r(ql, qr, ls, l, m) : q;
}

```

### 3.5 可持久化线段树/动态开点

- $\theta$ -index, 左闭右开  $[l, r)$ ; 按题目补全 `v` 和 `merge`, 0 号节点表示空树。
- `mdf(x,v,p)` 在版本根 `p` 上修改位置 `x`, 返回新根; `qry(l,r,p)` 查询版本 `p`。
- 多组测试每组先用 `seg={n}`; 清空节点池并设置长度。

```

#define ls(p) t[p].ls
#define rs(p) t[p].rs
struct SGT {
    struct V {};
    struct Node {
        int ls, rs;
    };
};

```

```

    V v;
};
int n;
vector<Node> t = {};
V merge(V a, V b) {}
void up(int p) {
    t[p].v = merge(t[ls(p)].v, t[rs(p)].v);
}
int mdf(int x, V v, int p = 0, int l = 0, int r = -1) {
    if (r < 0) r = n;
    t.push_back(t[p]);
    p = SZ(t) - 1;
    if (r - l == 1) {
        t[p].v = v;
        return p;
    }
    int m = (l + r) / 2;
    if (x < m) ls(p) = mdf(x, v, ls(p), l, m);
    else rs(p) = mdf(x, v, rs(p), m, r);
    up(p);
    return p;
}
V qry(int ql, int qr, int p = 0, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (ql <= l && r <= qr) return t[p].v;
    int m = (l + r) / 2;
    if (qr <= m) return qry(ql, qr, ls(p), l, m);
    if (m <= ql) return qry(ql, qr, rs(p), m, r);
    return merge(qry(ql, qr, ls(p), l, m), qry(ql, qr, rs(p), m, r));
}
};
#undef ls
#undef rs

```

```

SGT seg;
seg = {n};
vector<int> rt(q + 1);
rt[i] = seg.mdf(x, v, rt[k]);
auto ans = seg.qry(l, r, rt[i]);

```

- 若值域很大、只需动态开点而不保留历史版本，在上面的模板中新增 `node`，并将 `mdf` 替换为下列实现。
- `mdf` 的根 `p` 必传，遇到空节点时新建，并返回修改后的根；调用 `rt=seg.mdf(x,v,rt)`。
- `qry` 保持不变，调用时传入 `rt`；0 号节点表示空树。多组测试每组先用 `seg={n}`；清空，再令 `rt=0`。

```

int node() {
    t.push_back({});
    return SZ(t) - 1;
}

```

```

}
int mdf(int x, V v, int p, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (!p) p = node();
    if (r - l == 1) {
        t[p].v = v;
        return p;
    }
    int m = (l + r) / 2;
    if (x < m) ls(p) = mdf(x, v, ls(p), l, m);
    else rs(p) = mdf(x, v, rs(p), m, r);
    up(p);
    return p;
}

```

### 3.6 李超树

- 维护整数横坐标  $[0, n]$  上直线的最小值；0-index，左闭右开  $[l, r)$ 。
- `add(v,p)` 插入整条直线，`add(l,r,v,p)` 插入线段，均返回根；`merge(p,q)` 破坏性合并并返回根，之后不再使用 `q`；`qry(x,p)` 查询  $x$  处的最小值。
- $V\{k,b\}$  表示  $y = kx + b$ ；求最大值时将直线和答案同时取反。多组测试每组先用 `seg`  
 $\hookrightarrow =\{n\}$ ；清空，再令 `rt=0`。

```

#define ls(p) t[p].ls
#define rs(p) t[p].rs
struct SGT {
    static constexpr LL INF = (LL)4e18;
    struct V {
        LL k = 0, b = INF;
        I get(LL x) { return (I)k * x + b; }
    };
    struct Node {
        int ls, rs;
        V v;
    };
    int n;
    vector<Node> t = {};
    int node() {
        t.push_back({});
        return SZ(t) - 1;
    }
    int add(V v, int p, int l = 0, int r = -1) {
        if (r < 0) r = n;
        if (!p) p = node();
        int m = (l + r) / 2;
        if (v.get(m) < t[p].v.get(m)) swap(v, t[p].v);
        if (r - l == 1) return p;
        if (v.get(l) < t[p].v.get(l)) ls(p) = add(v, ls(p), l, m);
        else if (v.get(r - 1) < t[p].v.get(r - 1)) rs(p) = add(v, rs(p), m, r);
    }
}

```

```

    return p;
}
int add(int ql, int qr, V v, int p, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (ql <= l && r <= qr) return add(v, p, l, r);
    if (!p) p = node();
    int m = (l + r) / 2;
    if (ql < m) ls(p) = add(ql, qr, v, ls(p), l, m);
    if (m < qr) rs(p) = add(ql, qr, v, rs(p), m, r);
    return p;
}
int merge(int p, int q, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (!p || !q) return p | q;
    p = add(t[q].v, p, l, r);
    if (r - l == 1) return p;
    int m = (l + r) / 2;
    ls(p) = merge(ls(p), ls(q), l, m);
    rs(p) = merge(rs(p), rs(q), m, r);
    return p;
}
LL qry(int x, int p, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (!p) return INF;
    LL ans = clamp<I>(t[p].v.get(x), -INF, INF);
    if (r - l == 1) return ans;
    int m = (l + r) / 2;
    if (x < m) return min(ans, qry(x, ls(p), l, m));
    return min(ans, qry(x, rs(p), m, r));
}
};
#undef ls
#undef rs

```

### 3.7 线段树合并/分裂

- `split(l,r,p)` 返回的 `pair` 依次为剩余根和 `[l,r)` 的根。

```

int merge(int p, int q, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (!p || !q) return p | q;
    if (r - l == 1) {
        t[p].v = merge(t[p].v, t[q].v);
        return p;
    }
    int m = (l + r) / 2;
    ls(p) = merge(ls(p), ls(q), l, m);
    rs(p) = merge(rs(p), rs(q), m, r);
    up(p);
    return p;
}

```

```

}
pair<int, int> split(int ql, int qr, int p, int l = 0, int r = -1) {
    if (r < 0) r = n;
    if (!p || qr <= l || r <= ql) return {p, 0};
    if (ql <= l && r <= qr) return {0, p};
    int m = (l + r) / 2;
    int al = ls(p), ar = rs(p), bl = 0, br = 0;
    if (ql < m) tie(al, bl) = split(ql, qr, al, l, m);
    if (m < qr) tie(ar, br) = split(ql, qr, ar, m, r);
    ls(p) = al, rs(p) = ar;
    if (al || ar) up(p);
    else p = 0;
    if (!bl && !br) return {p, 0};
    int q = node();
    ls(q) = bl, rs(q) = br;
    up(q);
    return {p, q};
}

```

### 3.8 ODT

```

struct ODT {
    struct Node {
        int l, r, c;
        bool operator<(Node x) const {
            return l < x.l;
        }
    };
    set<Node> s;
    using IT = set<Node>::iterator;
    IT split(int x) {
        auto it = s.lower_bound({x});
        if (it != s.end() && it->l == x) return it;
        auto p = prev(it);
        if (p->r == x) return it;
        Node a = *p;
        s.erase(p);
        s.insert({a.l, x, a.c});
        return s.insert({x, a.r, a.c}).first;
    }
    IT merge(IT it) {
        Node a = *it;
        if (it != s.begin() && prev(it)->c == a.c) {
            a.l = prev(it)->l;
            s.erase(prev(it));
        }
        auto q = next(it);
        if (q != s.end() && q->c == a.c) {
            a.r = q->r;

```

```

    s.erase(q);
}
s.erase(it);
return s.insert(a).first;
}
void mdf(int l, int r, int c) {
    auto R = split(r), L = split(l);
    for (auto it = L; it != R; ++it) {
    }
    s.erase(L, R);
    merge(s.insert({l, r, c}).first);
}
};

```

```

ODT odt;
odt = {}; // 每组测试清空
odt.s.insert({0, n, c}); // 必须先完整覆盖 [0, n)
odt.mdf(l, r, x);

```

### 3.9 FHQ Treap

```

#define ls(p) t[p].ch[0]
#define rs(p) t[p].ch[1]
struct FHQTreap {
    struct V {
    };
    struct Node {
        int ch[2]{}; sz = 0;
        ULL pri = 0;
        V v;
        bool rev = false;
    };
    vector<Node> t = {};
    int rt = 0;
    int size(int p) { return t[p].sz; }
    int size() { return size(rt); }
    void up(int p) {
        t[p].sz = size(ls(p)) + size(rs(p)) + 1;
    }
    void apply_rev(int p) {
        if (p) swap(ls(p), rs(p)), t[p].rev ^= 1;
    }
    void down(int p) {
        if (!t[p].rev) return;
        apply_rev(ls(p)), apply_rev(rs(p));
        t[p].rev = false;
    }
    int node(V v) {
        t.push_back({}, 1, rnd(), v);
    }
};

```

```

return SZ(t) - 1;
}
void split(int p, int k, int& x, int& y) {
    if (!p) return x = y = 0, void();
    down(p);
    if (k <= size(ls(p))) y = p, split(ls(p), k, x, ls(y));
    else x = p, split(rs(p), k - size(ls(p)) - 1, rs(x), y);
    up(p);
}
int merge(int p, int q) {
    if (!p || !q) return p | q;
    if (t[p].pri < t[q].pri) {
        down(p), rs(p) = merge(rs(p), q), up(p);
        return p;
    }
    down(q), ls(q) = merge(p, ls(q)), up(q);
    return q;
}
int insert(int p, int k, int q) {
    if (!p) return q;
    if (t[q].pri < t[p].pri) {
        split(p, k, ls(q), rs(q)), up(q);
        return q;
    }
    down(p);
    if (k <= size(ls(p))) ls(p) = insert(ls(p), k, q);
    else rs(p) = insert(rs(p), k - size(ls(p)) - 1, q);
    up(p);
    return p;
}
void insert(int k, V v) {
    rt = insert(rt, k, node(v));
}
int erase(int p, int k) {
    down(p);
    int s = size(ls(p));
    if (k == s) return merge(ls(p), rs(p));
    if (k < s) ls(p) = erase(ls(p), k);
    else rs(p) = erase(rs(p), k - s - 1);
    up(p);
    return p;
}
void erase(int k) {
    rt = erase(rt, k);
}
void reverse(int l, int r) {
    int x, y, z;
    split(rt, r, x, z), split(x, l, x, y);
    apply_rev(y), rt = merge(merge(x, y), z);
}

```

```

int kth(int p, int k) {
    down(p);
    int s = size(ls(p));
    if (k == s) return p;
    if (k < s) return kth(ls(p), k);
    return kth(rs(p), k - s - 1);
}
void clear() {
    t.assign(1, {}), rt = 0;
}
};
#undef ls
#undef rs

```

```

FHQTreap tr;
for (int i = 0; i < n; i++) tr.insert(i, {}); // 初始化
tr.clear(); // 多测清空

```

### 3.10 pbds tree

- `tr.order_of_key(x)` 返回严格小于 `x` 的元素个数。
- `tr.find_by_order(k)` 返回 `0`-index 第 `k` 小元素的迭代器；若 `k`  $\geq$  `tr.size()`，返回 `tr.end()`。

```

#include <bits/extc++.h>
using namespace __gnu_pbds;
template <class T>
using Tree = tree<T, null_type, std::less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

```

### 3.11 普通莫队

查询使用 `0`-index，左闭右开  $[l, r)$ ；

```

struct Q {
    int l, r, id;
};
vector<Q> q;
void solve(int n) {
    int B = max(1, int(sqrt(n)));
    sort(ALL(q), [&](Q a, Q b) {
        int x = a.l / B, y = b.l / B;
        if (x != y) return x < y;
        return x & 1 ? a.r > b.r : a.r < b.r;
    });
    int l = 0, r = 0;
    for (auto [ql, qr, id] : q) {
        while (ql < l) add(--l);
        while (r < qr) add(r++);
    }
}

```

```

while (l < ql) del(l++);
while (qr < r) del(--r);
answer(id);
}
}

```

### 3.12 带修莫队

查询使用 `0`-index，左闭右开  $[l, r)$ 。将修改次数作为第三维。

```

struct V {
};
struct Q {
    int l, r, t, id;
};
struct C {
    int p;
    V v;
};
vector<V> a;
vector<Q> q;
vector<C> c;
void modify(int p, V v) {
    c.push_back({p, v});
}
void ask(int l, int r) {
    q.push_back({l, r, SZ(c), SZ(q)});
}
void apply(int k, int l, int r) {
    auto& [p, v] = c[k];
    if (l <= p && p < r) del(p);
    swap(a[p], v);
    if (l <= p && p < r) add(p);
}
void solve() {
    int B = max(1, int(pow(SZ(a), 2.0 / 3)));
    sort(ALL(q), [&](Q x, Q y) {
        int bx = x.l / B, by = y.l / B;
        if (bx != by) return bx < by;
        bx = x.r / B, by = y.r / B;
        if (bx != by) return bx < by;
        return x.t < y.t;
    });
    int l = 0, r = 0, t = 0;
    for (auto [ql, qr, qt, id] : q) {
        while (t < qt) apply(t++, l, r);
        while (qt < t) apply(--t, l, r);
        while (ql < l) add(--l);
        while (r < qr) add(r++);
        while (l < ql) del(l++);
    }
}

```

```

    while (qr < r) del(--r);
    answer(id);
}
}

```

### 3.13 回滚莫队

适用于好加难删。按左端点分块，块内右端点单调增加。左端点临时加入，同块询问暴力处理，回答后均撤销。查询使用  $\theta$ -index，左闭右开  $[l, r)$ 。补全 add、snapshot、rollback、answer，其中每次 add 的全部影响都必须可撤销。复杂度为  $O((n+q)\sqrt{n}A)$ ；好删难加时反向处理。

```

struct Q {
    int l, r, id;
};
vector<Q> q;
void ask(int l, int r) {
    q.push_back({l, r, SZ(q)});
}
void solve(int n) {
    int B = max(1, int(sqrt(n)));
    sort(ALL(q), [&](Q a, Q b) {
        int x = a.l / B, y = b.l / B;
        if (x != y) return x < y;
        return a.r < b.r;
    });
    for (int i = 0; i < SZ(q); i) {
        int b = q[i].l / B, j = i;
        while (j < SZ(q) && q[j].l / B == b) j++;
        int R = min(n, (b + 1) * B), r = R;
        int base = snapshot();
        for (int k = i; k < j; k++) {
            auto [ql, qr, id] = q[k];
            if (qr <= R) {
                int t = snapshot();
                for (int p = ql; p < qr; p++) add(p);
                answer(id), rollback(t);
                continue;
            }
            while (r < qr) add(r++);
            int t = snapshot();
            for (int p = R - 1; p >= ql; p--) add(p);
            answer(id), rollback(t);
        }
        rollback(base), i = j;
    }
}

```

### 3.14 莫队二次离线

一种能够将  $n\sqrt{n}\log n$  的莫队变为优美的  $n\sqrt{n}$  的 trick。由于直接讲可能有点玄乎，我们以区间逆序对数为例讲解莫队二次离线的过程。

可以发现，对于区间逆序对而言，当我们向右移动右指针时，相当于计算有多少个  $[l, r]$  中的数  $> a_{r+1}$ ，而如果我们把一段连续的移动左右端点的序列看作一次操作，那么显然操作总次数是  $O(q)$  的，因此我们只要求出每次“操作”会使得逆序对数变化多少，我们就可以求出每个时刻的逆序对数进而求出答案。现在考虑如何求这个增量，以右端点向右移为例，假设右端点从  $r_1$  移到  $r_2$ ，此时左端点为  $l$ ，记  $f(l, r, x)$  变为  $[l, r]$  中有多少个  $> a_x$  的数，那么相当于求  $\sum_{i=r_1+1}^{r_2} f(l, i-1, i)$ ，差分一下可以变

成  $\sum_{i=r_1+1}^{r_2} f(l, i-1, i) - f(l, l-1, i)$ ；前面一部分就维护  $f(l, i-1, i)$  的前缀和即可，后一部分我们在  $l-1$  处加入三元组  $(l, r_1, r_2)$  然后扫描线，扫到  $x$  时候考虑所有三元组  $(l, r_1, r_2)$ ，由于  $\sum r_2 - r_1$  是  $n\sqrt{n}$  级别的，因此可以暴力枚举区间中所有位置，这样相当于  $O(n)$  次单点加  $O(n\sqrt{n})$  次区间查询。分块维护即可。

## 4 Tree

### 4.1 LCA

```

int dn, dfn[N], out[N], fa[N], dep[N], mi[__lg(N) + 1][N];
vector<int> ch[N];
int mn(int u, int v) {
    return dfn[u] < dfn[v] ? u : v;
}
void dfs_lca(int u, int f) {
    fa[u] = f, dep[u] = u == f ? 0 : dep[f] + 1;
    mi[0][dfn[u]] = dn++;
    for (int v : G[u]) if (v != f) ch[u].pb(v), dfs_lca(v, u);
    out[u] = dn;
}
void init_lca(int r) {
    dn = 0, dfs_lca(r, r);
    for (int j = 1; (1 << j) <= dn; j++)
        for (int i = 0; i + (1 << j) <= dn; i++)
            mi[j][i] = mn(mi[j-1][i], mi[j-1][i + (1 << (j-1))]);
}
int lca(int u, int v) {
    if (u == v) return u;
    if ((u = dfn[u]) > (v = dfn[v])) swap(u, v);
    int k = __lg(v - u++);
    return mn(mi[k][u], mi[k][v - (1 << k) + 1]);
}
bool anc(int u, int v) {
    return dfn[u] <= dfn[v] && dfn[v] < out[u];
}

```

```
int findchild(int u, int v) {
    assert(u ≠ v);
    if (!anc(u, v)) return fa[u];
    return *prev(upper_bound(ALL(ch[u]), v, [](int x, int y) {
        return dfn[x] < dfn[y];
    }));
}
```

## 4.2 点分治

```
int sz[N], cfa[N];
bool ban[N];
vector<int> CT[N];
int getsz(int u, int f) {
    sz[u] = 1;
    for (int v : G[u]) if (v ≠ f && !ban[v]) sz[u] += getsz(v, u);
    return sz[u];
}
int getcen(int u, int f, int n) {
    for (int v : G[u])
        if (v ≠ f && !ban[v] && sz[v] > n / 2) return getcen(v, u, n);
    return u;
}
int divide(int u, int f = -1) {
    int c = getcen(u, -1, getsz(u, -1));
    cfa[c] = f;
    if (f ≠ -1) CT[f].pb(c);
    work(c), ban[c] = 1;
    for (int v : G[c]) if (!ban[v]) divide(v, c);
    return c;
}
```

## 4.3 虚树

```
vector<int> VT[N];
vector<int> virtual_tree(vector<int> a) {
    auto cmp = [](int u, int v) { return dfn[u] < dfn[v]; };
    sort(ALL(a), cmp);
    int k = SZ(a);
    for (int i = 1; i < k; i++) a.pb(lca(a[i - 1], a[i]));
    sort(ALL(a), cmp), a.erase(unique(ALL(a)), a.end());
    for (int u : a) VT[u].clear();
    for (int i = 1; i < SZ(a); i++) VT[lca(a[i - 1], a[i])].pb(a[i]);
    return a;
}
```

## 4.4 树 Hash

```
ULL salt = rnd();
ULL mix(ULL x) {
    x += salt;
    x ^= x << 13;
    x ^= x >> 7;
    return x ^ (x << 17);
}
ULL tree_hash(int u, int f = -1) {
    ULL h = 1;
    for (int v : G[u]) if (v ≠ f) h += mix(tree_hash(v, u));
    return h;
}
```

# 5 String

## 5.1 String Hash

```
using H = pair<LL, LL>;
struct StringHash {
    static constexpr LL P1 = 917120411, P2 = 1000000009;
    static constexpr LL B1 = 19260817, B2 = 19491001;
    int n;
    vector<LL> pw1, pw2, h1, h2;
    StringHash(const string& s) : n(SZ(s)),
        pw1(n + 1, 1), pw2(n + 1, 1), h1(n + 1), h2(n + 1) {
        for (int i = 0; i < n; i++) {
            int x = s[i] + 1;
            pw1[i + 1] = pw1[i] * B1 % P1;
            pw2[i + 1] = pw2[i] * B2 % P2;
            h1[i + 1] = (h1[i] * B1 + x) % P1;
            h2[i + 1] = (h2[i] * B2 + x) % P2;
        }
    }
    H get(int l, int r) {
        assert(0 ≤ l && l ≤ r && r ≤ n);
        LL x = (h1[r] - h1[l] * pw1[r - l] % P1 + P1) % P1;
        LL y = (h2[r] - h2[l] * pw2[r - l] % P2 + P2) % P2;
        return {x, y};
    }
};
```

## 5.2 KMP

```
vector<int> kmp(string s) {
    int n = SZ(s);
    vector<int> f(n + 1);
    for (int i = 1, j = 0; i < n; i++) {
        while (j && s[i] ≠ s[j]) j = f[j];
```

```

    if (s[i] == s[j]) j++;
    f[i + 1] = j;
}
return f;
}

```

## 5.3 Z 函数

```

vector<int> ZFunc(string s) {
    int n = SZ(s), l = 0, r = 0;
    vector<int> z(n);
    if (n) z[0] = n;
    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = min(z[i - l], r - i + 1);
        while (i + z[i] < n && s[i + z[i]] == s[z[i]]) z[i]++;
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }
    return z;
}

vector<int> exKMP(string s, string t, vector<int> z) {
    int n = SZ(s), m = SZ(t), l = 0, r = -1;
    vector<int> p(n);
    for (int i = 0; i < n; i++) {
        if (i <= r) p[i] = min(z[i - l], r - i + 1);
        while (i + p[i] < n && p[i] < m && s[i + p[i]] == t[p[i]]) p[i]++;
        if (i + p[i] - 1 > r) l = i, r = i + p[i] - 1;
    }
    return p;
}

```

## 5.4 AC 自动机

```

int ins(const string& s) {
    int p = 0;
    for (char x : s) {
        int c = x - 'a';
        if (!tr[p][c]) tr[p][c] = ++tot;
        p = tr[p][c];
    }
    return p;
}

void get_f() {
    queue<int> q;
    for (int c = 0; c < 26; c++) if (tr[0][c]) q.push(tr[0][c]);
    while (SZ(q)) {
        int u = q.front();
        q.pop();
        for (int c = 0; c < 26; c++) {
            int v = tr[u][c];

```

```

        if (v) fail[v] = tr[fail[u]][c], q.push(v);
        else tr[u][c] = tr[fail[u]][c];
    }
}
}

```

## 5.5 SA

- $sa[i]$  是排名第  $i$  的后缀起点,  $rk[i]$  是后缀  $s[i, n)$  的排名, 均为 0-index。
- $ht[i]$  等于  $\text{lcp}(s[sa[i-1], n), s[sa[i], n))$ , 其中  $ht[0]=0$ 。

```

struct SA {
    vector<int> sa, rk, ht;
    SA(const string& s) : sa(SZ(s)), rk(SZ(s)), ht(SZ(s)) {
        int n = SZ(s);
        if (!n) return;
        iota(ALL(sa), 0);
        sort(ALL(sa), [&](int a, int b) { return s[a] < s[b]; });
        for (int i = 1; i < n; i++) rk[sa[i]] = rk[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
        vector<int> old(n), cnt(n);
        for (int w = 1; rk[sa.back()] < n - 1; w *= 2) {
            int m = 0;
            for (int i = n - w; i < n; i++) old[m++] = i;
            for (int x : sa) if (x >= w) old[m++] = x - w;
            fill(ALL(cnt), 0);
            for (int x : rk) cnt[x]++;
            partial_sum(ALL(cnt), cnt.begin());
            for (int i = n; i--;) sa[--cnt[rk[old[i]]]] = old[i];
            swap(rk, old);
            rk[sa[0]] = 0;
            for (int i = 1; i < n; i++) {
                int a = sa[i - 1], b = sa[i];
                rk[b] = rk[a] + (old[a] != old[b]
                    || (a + w < n ? old[a + w] : -1) != (b + w < n ? old[b + w] : -1));
            }
        }
        for (int i = 0, k = 0; i < n; i++) {
            if (!rk[i]) { k = 0; continue; }
            if (k) k--;
            int j = sa[rk[i] - 1];
            while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
            ht[rk[i]] = k;
        }
    }
};

```

## 5.6 SAM

- 根节点编号为 1; 普通字符串从  $p=1$  开始, 依次令  $p=\text{ins}(p, c - 'a')$ 。

- 最多产生  $2n$  个节点；fa 是后缀链接，len 是状态所表示子串的最大长度。

```
vector<array<int, 26>> ch(2 * n + 2);
vector<int> fa(2 * n + 2), len(2 * n + 2);
int tot = 1;
auto ins = [&](int p, int c) {
    int cur = ++tot;
    len[cur] = len[p] + 1;
    for (; p && !ch[p][c]; p = fa[p]) ch[p][c] = cur;
    if (!p) fa[cur] = 1;
    else {
        int q = ch[p][c];
        if (len[q] == len[p] + 1) fa[cur] = q;
        else {
            int cl = ++tot;
            ch[cl] = ch[q], fa[cl] = fa[q], len[cl] = len[p] + 1;
            for (; p && ch[p][c] == q; p = fa[p]) ch[p][c] = cl;
            fa[q] = fa[cur] = cl;
        }
    }
    return cur;
};
```

## 5.7 Manacher

- 对应原串半开区间为  $[(i-p[i]+1)/2, (i+p[i]-1)/2)$ 。

```
vector<int> Manacher(const string& s) {
    string t = "#";
    for (char c : s) t += c, t += '#';
    int n = t.size();
    vector<int> p(n);
    for (int i = 0, l = 0, r = 0; i < n; i++) {
        p[i] = i < r ? min(p[2 * l - i], r - i) : 0;
        while (i + p[i] < n && i - p[i] >= 0)
            && t[i + p[i]] == t[i - p[i]] p[i]++;
        if (i + p[i] > r) l = i, r = i + p[i];
    }
    return p;
}
```

## 5.8 PAM

- sz-2 是本质不同回文串数，num[las] 是当前回文后缀数，tot 是已经插入串的回文子串总数。
- 全部插入后调用一次 calc()，即可由 cnt[p] 得到每个回文串的出现次数。

```
struct PAM {
    vector<array<int, 26>> ch;
    vector<int> fail, len, cnt, num, s;
```

```
int sz = 2, las = 0;
LL tot = 0;
PAM(int n) : ch(n + 2), fail(n + 2), len(n + 2),
             cnt(n + 2), num(n + 2), s(1, -1) {
    fail[0] = fail[1] = 1, len[1] = -1;
}
int add(int c) {
    s.pb(c);
    int n = SZ(s) - 1, p = las;
    while (s[n - len[p] - 1] != c) p = fail[p];
    if (!ch[p][c]) {
        int q = fail[p], x = sz++;
        while (s[n - len[q] - 1] != c) q = fail[q];
        len[x] = len[p] + 2, fail[x] = ch[q][c];
        num[x] = num[fail[x]] + 1, ch[p][c] = x;
    }
    las = ch[p][c], cnt[las]++, tot += num[las];
    return las;
}
void calc() {
    for (int p = sz - 1; p > 1; p--) cnt[fail[p]] += cnt[p];
}
};
```

## 6 Math

### 6.1 ExGCD

- exgcd(a,b,x,y) 要求  $a, b \geq 0$  且不同时为零，返回  $g = \gcd(a, b)$ ，并满足  $ax + by = g$ 。
- linear\_equation(a,b,c) 要求  $a, b$  不同时为零。无解返回 nullopt；否则返回  $\{x_0, y_0, \rightarrow dx, dy\}$ ，通解为  $(x, y) = (x_0 + kdx, y_0 + kdy)$ ,  $k \in \mathbb{Z}$ 。
- linear\_congruence(a,b,m) 求  $ax \equiv b \pmod{m}$ 。无解返回 nullopt；否则返回  $\{r, p\}$ ，全部解为  $x \equiv r \pmod{p}$ 。

```
LL exgcd(LL a, LL b, LL& x, LL& y) {
    assert(a >= 0 && b >= 0 && (a || b));
    if (!b) return x = 1, y = 0, a;
    LL g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}
```

```
optional<array<LL, 4>> linear_equation(LL a, LL b, LL c) {
    assert(a || b);
    LL x, y, g = exgcd(abs(a), abs(b), x, y);
    if (c % g) return nullopt;
    x *= c / g, y *= c / g;
    if (a < 0) x = -x;
    if (b < 0) y = -y;
```

```

    return array<LL, 4>{x, y, b / g, -a / g};
}
optional<pair<LL, LL>> linear_congruence(LL a, LL b, LL m) {
    assert(m > 0);
    a %= m;
    b %= m;
    if (a < 0) a += m;
    if (b < 0) b += m;
    LL x, y, g = exgcd(a, m, x, y);
    if (b % g) return nullopt;
    LL p = m / g;
    LL r = I(x) * (b / g) % p;
    if (r < 0) r += p;
    return pair<LL, LL>{LL(r), p};
}

```

## 6.2 BSGS / ExBSGS

- 求最小的  $x \geq 0$  使  $a^x \equiv b \pmod{m}$ ; bsgs 要求  $\gcd(a, m) = 1$ , exbsgs 无此限制, 无解返回 -1。
- 时间与空间复杂度均为  $O(\sqrt{m})$ , 使用哈希表保存 baby steps。

```

LL qp(LL a, LL b, LL m) {
    LL r = 1 % m;
    for (; b; b >>= 1, a = LL(I(a) * a % m))
        if (b & 1) r = LL(I(r) * a % m);
    return r;
}
LL bsgs(LL a, LL b, LL m, LL k = 1) {
    assert(m > 1 && gcd(a, m) == 1);
    if (k == b) return 0;
    LL n = LL(sqrtl((long double)m)) + 1, e = b;
    unordered_map<LL, LL> mp;
    for (LL j = 0; j < n; j++, e = LL(I(e) * a % m)) mp[e] = j;
    a = qp(a, n, m), e = k;
    for (LL i = 1; i <= n; i++) {
        e = LL(I(e) * a % m);
        auto it = mp.find(e);
        if (it != mp.end()) return LL(I(i) * n - it->second);
    }
    return -1;
}
LL exbsgs(LL a, LL b, LL m) {
    assert(m > 0);
    if (m == 1) return 0;
    a %= m, b %= m;
    if (a < 0) a += m;
    if (b < 0) b += m;
    if (b == 1) return 0;

```

```

LL k = 1, c = 0, g;
while ((g = gcd(a, m)) > 1) {
    if (b % g) return -1;
    b /= g, m /= g, k = LL(I(k) * (a / g) % m), c++;
    if (k == b) return c;
}
LL x = bsgs(a, b, m, k);
return x < 0 ? -1 : x + c;
}

```

## 6.3 线性筛

- sieve(n) 只调用一次,  $O(n)$  求出不超过  $n$  的全部质数、最小质因子 mn、欧拉函数 phi 和莫比乌斯函数 mu。

```

vector<int> pr;
int mn[N], phi[N], mu[N];
void sieve(int n) {
    phi[1] = mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!mn[i]) pr.pb(i), mn[i] = i, phi[i] = i - 1, mu[i] = -1;
        for (int p : pr) {
            if (p > n / i) break;
            int x = i * p; mn[x] = p;
            if (i % p == 0) { phi[x] = phi[i] * p; break; }
            phi[x] = phi[i] * (p - 1), mu[x] = -mu[i];
        }
    }
}

```

## 6.4 Miller-Rabin / Pollard-Rho

- 依赖前面的 qp(a,b,m) 与 Basic 中的 rnd; prime(n) 对正 LL 确定性判素。
- 复杂度 (定长整数): Miller-Rabin 为  $O(k \log n)$  (这里  $k = 7$ ); Pollard-Rho 找因子的期望为  $O(\sqrt{p}) \leq O(n^{1/4})$ , 空间  $O(1)$ , 其中  $p$  为最小质因子。
- factor(n) 返回带重数且升序排列的质因子, factor(1) 返回空数组。

```

bool prime(LL n) {
    if (n < 4) return n > 1;
    if (n % 2 == 0) return false;
    int s = __builtin_ctzll(n - 1);
    LL d = (n - 1) >> s;
    for (LL a : {2LL, 325LL, 9375LL, 28178LL,
        450775LL, 9780504LL, 1795265022LL}) {
        if (a % n == 0) continue;
        LL x = qp(a, d, n);
        if (x == 1 || x == n - 1) continue;
        for (int r = 1; r < s && x != n - 1; r++) x = LL(I(x) * x % n);
        if (x != n - 1) return false;
    }

```

```

}
return true;
}
LL rho(LL n) {
    if (n % 2 == 0) return 2;
    while (true) {
        LL c = rnd() % (n - 1) + 1, x = rnd() % n, y = x, d = 1;
        auto f = [&](LL x) { return LL((I(x) * x + c) % n); };
        while (d == 1) x = f(x), y = f(f(y)), d = gcd(abs(x - y), n);
        if (d < n) return d;
    }
}
void fac(LL n, vector<LL>& a) {
    if (n == 1) return;
    if (prime(n)) return a.pb(n);
    LL d = rho(n);
    fac(d, a), fac(n / d, a);
}
vector<LL> factor(LL n) {
    assert(n > 0);
    vector<LL> a;
    fac(n, a), sort(ALL(a));
    return a;
}

```

## 6.5 Primitive Root

- 求质数  $p$  的最小正原根，依赖前一个 Miller-Rabin / Pollard-Rho 条目。
- 这里只处理质数模数；primitive\_root(2)=1。

```

LL primitive_root(LL p) {
    assert(prime(p));
    if (p == 2) return 1;
    vector<LL> f = factor(p - 1);
    f.erase(unique(ALL(f)), f.end());
    for (LL g = 2; g < p; g++) {
        bool ok = true;
        for (LL q : f) {
            if (qp(g, (p - 1) / q, p) == 1) {
                ok = false;
                break;
            }
        }
        if (ok) return g;
    }
    return 0;
}

```

## 6.6 exCRT

- exCRT( $r, m$ ) 依赖 linear\_congruence，合并任意个方程  $x \equiv r_i \pmod{m_i}$ ，模数不要求互质，但必须为正数。
- 返回  $\{r, m\}$  表示全部解为  $x \equiv r \pmod{m}$ ；无解返回  $\{0, 0\}$ 。要求最终模数最小公倍数不超过 LL。

```

pair<LL, LL> exCRT(vector<LL> r, vector<LL> m) {
    assert(SZ(r) == SZ(m));
    LL R = 0, M = 1;
    for (int i = 0; i < SZ(r); i++) {
        assert(m[i] > 0);
        auto z = linear_congruence(M, LL((I(r[i]) - R) % m[i]), m[i]);
        if (!z) return {0, 0};
        auto [x, p] = *z;
        I nm = I(M) * p;
        assert(nm <= LLONG_MAX);
        R = LL(I(R) + I(M) * x), M = LL(nm);
    }
    return {R, M};
}

```

## 6.7 二次剩余

- 依赖前面的 qp(a,b,m) 和 Basic 中的 I；mod\_sqrt(a,p) 要求  $p$  为质数，返回较小的平方根，二次非剩余返回 -1。
- 另一根为  $p - r$ ，复杂度为  $O(\log^2 p)$ 。

```

LL mod_sqrt(LL a, LL p) {
    assert(p >= 2);
    a %= p;
    if (a < 0) a += p;
    if (!a || p == 2) return a;
    if (qp(a, (p - 1) / 2, p) != 1) return -1;
    int s = __builtin_ctzll(p - 1); LL q = (p - 1) >> s, z = 2;
    while (qp(z, (p - 1) / 2, p) != p - 1) z++;
    LL c = qp(z, q, p), x = qp(a, (q + 1) / 2, p), t = qp(a, q, p);
    for (int m = s; t != 1; t = qp(a, q, p)) {
        int i = 0;
        for (LL y = t; y != 1; i++) y = LL(I(y) * y % p);
        LL b = qp(c, 1LL << (m - i - 1), p);
        x = LL(I(x) * b % p), c = LL(I(b) * b % p);
        t = LL(I(t) * c % p), m = i;
    }
    return min(x, p - x);
}

```

## 6.8 floor sum

floor\_sum(n,m,a,b) 计算

$$\sum_{i=0}^{n-1} \left\lfloor \frac{ai+b}{m} \right\rfloor.$$

要求  $n, a, b \geq 0, m > 0$  且答案不超过 LL, 复杂度为  $O(\log \max(a, m))$ 。

```
LL floor_sum(LL n, LL m, LL a, LL b) {
    LL ans = 0;
    for (;;) {
        if (a >= m) ans += LL(I(n) * (n - 1) / 2 * (a / m)), a %= m;
        if (b >= m) ans += LL(I(n) * (b / m)), b %= m;
        I y = I(a) * n + b;
        if (y < m) return ans;
        n = LL(y / m), b = LL(y % m), swap(a, m);
    }
}
```

## 6.9 万能欧几里得

euclid(a,b,c,n,U,R) 要求  $a, b, n \geq 0, c > 0$ , 对直线  $y = (ax+b)/c$  ( $0 < x \leq n$ ) 生成操作序列: 每上穿一条横线记 U, 每右穿一条竖线记 R, 经过整点时先 U 后 R。返回序列中元素按顺序用 + 合并的结果, 复杂度为  $O(\log \max(a, c) + \log(b/c + 1))$  次合并。

类型 T 的默认构造须为幺元, + 须满足结合律且保留顺序。字符串调试可用 euclid  $\hookrightarrow$  (a,b,c,n,string("U"),string("R")); 实际使用时令 T 保存所求统计量, 并在 operator+ 中合并两段贡献。

例如求  $\sum_{i=1}^n \lfloor (ai+b)/c \rfloor$ , 可令 T={x,y,s} 分别记录 R 数、U 数及答案, 合并公式为

$$(x_1, y_1, s_1) + (x_2, y_2, s_2) = (x_1 + x_2, y_1 + y_2, s_1 + s_2 + x_2 y_1),$$

并取  $U=\{0,1,0\}, R=\{1,0,0\}$ , 返回值的 s 即为答案。

```
template<class T>
T pw(T a, LL n) {
    T r{};
    for (; n >= 1, a = a + a) if (n & 1) r = r + a;
    return r;
}

template<class T>
T euclid(LL a, LL b, LL c, LL n, T U, T R) {
    if (b >= c) return pw(U, b / c) + euclid(a, b % c, c, n, U, R);
    if (a >= c) return euclid(a % c, b, c, n, U, pw(U, a / c) + R);
    LL m = LL((I(a) * n + b) / c);
    if (!m) return pw(R, n);
    return pw(R, (c - b - 1) / a) + U
        + euclid(c, (c - b - 1) % a, a, m - 1, R, U)
        + pw(R, n - LL((I(c) * m - b - 1) / a));
}
```

## 6.10 杜教筛

狄利克雷卷积定义为  $(f * g)(n) = \sum_{d|n} f(d)g(n/d)$ 。设  $f * g = h$ ,  $F, G, H$  分别为  $f, g, h$  的前缀和, 则

$$H(n) = \sum_{d=1}^n g(d)F(\lfloor n/d \rfloor).$$

当  $g(1) = 1$  时可分离出  $F(n)$ , 再对  $\lfloor n/d \rfloor$  数论分块。代码中 G(n)、H(n) 分别返回  $g, h$  的前缀和, sf 保存预处理范围内  $F$ , du(n) 返回  $F(n)$ 。取预处理上界  $B \approx n^{2/3}$  时, 单次复杂度为  $O(n^{2/3})$ 。

默认求莫比乌斯函数前缀和: 先保证 sieve(B) 已完成, 再调用 initDu(B)。求欧拉函数前缀和时, 将 initDu 中的 mu[i] 改为 phi[i], 并令  $H(n)=n(n+1)/2$ 。所有中间结果须在 LL 范围内。

```
vector<LL> sf;
unordered_map<LL, LL> mem;
LL G(LL n) { return n; }
LL H(LL) { return 1; }
void initDu(int n) {
    sf.assign(n + 1, 0), mem.clear();
    for (int i = 1; i <= n; i++) sf[i] = sf[i - 1] + mu[i];
}
LL du(LL n) {
    if (n < SZ(sf)) return sf[n];
    if (auto it = mem.find(n); it != mem.end()) return it->second;
    LL ans = H(n);
    for (LL l = 2, r; l <= n; l = r + 1) {
        LL q = n / l;
        r = n / q;
        ans -= (G(r) - G(l - 1)) * du(q);
    }
    return mem[n] = ans;
}
```

## 6.11 Min\_25 筛

求积性函数  $f$  的前缀和模 P。要求  $f(p) = c_0 + c_1 p + c_2 p^2$ , 且  $f(p^e)$  能快速计算; 分别写在 c 和 fpk(p,e) 中。代码默认  $f = \varphi$ , 故 min25(n) 返回  $\sum_{i=1}^n \varphi(i) \bmod P$ 。

使用前通过前面的线性筛一次性求出  $\sqrt{n}$  内的质数。时间复杂度为  $O(n^{3/4}/\log n)$ , 空间复杂度为  $O(\sqrt{n})$ 。

```
using A = array<LL, 3>;
A c{P - 1, 1, 0};
LL fpk(LL p, int e) { return Mul(qp(p % P, e - 1), (p - 1) % P); }
LL min25(LL n) {
    int B = sqrtl(n), m = upper_bound(ALL(pr), B) - pr.begin();
    vector<LL> w; vector<A> g;
```

```

LL iv2 = qp(2), iv6 = qp(6);
auto sum = [&](LL n) {
    LL x = n % P;
    return A{(n - 1) % P,
              (Mul(Mul(x, (x + 1) % P), iv2) - 1 + P) % P,
              (Mul(Mul(Mul(x, (x + 1) % P), (2 * x + 1) % P), iv6) - 1 + P) % P};
};
for (LL l = 1, r; l <= n; l = r + 1) {
    LL x = n / l; r = n / x;
    w.pb(x), g.pb(sum(x));
}
auto id = [&](LL x) { return x <= B ? SZ(w) - x : n / x - 1; };
for (int j = 0; j < m; j++) {
    LL p = pr[j], x = p % P;
    A pk{1, x, Mul(x, x)};
    int t = id(p - 1);
    for (int i = 0; I(p) * p <= w[i]; i++) {
        int k = id(w[i] / p);
        for (int d = 0; d < 3; d++) {
            LL &v = g[i][d], z = (g[k][d] - g[t][d] + P) % P;
            v = (v - Mul(pk[d], z) + P) % P;
        }
    }
}
auto S = [&](auto&& self, LL x, int y) -> LL {
    if (y && pr[y - 1] >= x) return 0;
    int k = id(x), t = id(y ? pr[y - 1] : 1); LL ans = 0;
    for (int d = 0; d < 3; d++)
        ans = Add(ans, Mul(c[d], (g[k][d] - g[t][d] + P) % P));
    for (int i = y; i < m && I(pr[i]) * pr[i] <= x; i++) {
        LL pe = pr[i];
        for (int e = 1;; e++) {
            LL z = self(self, x / pe, i + 1);
            if (e > 1) z = Add(z, 1);
            ans = Add(ans, Mul(fpk(pr[i], e), z));
            if (pe > x / pr[i]) break;
            pe *= pr[i];
        }
    }
    return ans;
};
return Add(S(S, n, 0), 1);
}

```

## 7 Geometry

### 7.1 vector

$\arg(a,b)$  要求  $a, b$  均为非零向量，返回从  $a$  到  $b$  的有向夹角，范围为  $[-\pi, \pi]$ ；正值表示逆时针，负值表示顺时针，0 表示同向， $\pm\pi$  表示反向。无向最小夹角为  $\text{abs}(\arg \hookrightarrow (a,b))$ 。

$\text{dir}(a,b)$  同样要求两向量非零，返回 1 表示同向，-1 表示反向，0 表示不平行。仅判断转向时直接使用  $\text{sgn}(a\%b)$ ； $\arg$  在反向处存在  $-\pi/\pi$  断点。

```

using db = long double;
const db eps = 1e-10;
int sgn(db x) { return x < -eps ? -1 : x > eps; }
bool eq(db x, db y) { return !sgn(x - y); }
struct p2 {
    db x, y;
    db len() { return hypot(x, y); }
    p2 operator+(p2 b) { return {x + b.x, y + b.y}; }
    p2 operator-(p2 b) { return {x - b.x, y - b.y}; }
    p2 operator*(db k) { return {x * k, y * k}; }
    p2 operator/(db k) { return {x / k, y / k}; }
    db operator*(p2 b) { return x * b.x + y * b.y; }
    db operator%(p2 b) { return x * b.y - y * b.x; }
};
using ps = vector<p2>;
db arg(p2 a, p2 b) { return atan2(a % b, a * b); }
int dir(p2 a, p2 b) { return sgn(a % b) ? 0 : sgn(a * b); }

```

### 7.2 极角排序

以下函数均要求向量非零。 $\text{half}$  将极角按  $[0, \pi)$  与  $[\pi, 2\pi)$  分成两半；默认启用精确版，浮点数据需要把坐标轴附近视为同一方向时，可注释精确版并启用  $\text{eps}$  版。

$\text{cmp}(a,b)$  判断  $a$  的极角是否小于  $b$ ，可直接用于  $\text{sort}$ ； $\text{cmp\_eq}(a,b)$  判断两向量是否同向，不比较长度。

$\text{cmp\_ct}(a,b,c)$  判断圆周上的方向顺序是否为逆时针  $a \rightarrow b \rightarrow c$ ：成立返回 1，不成立返回 0， $a, c$  中任一向量与  $b$  同向时返回 -1。

```

int half(p2 a) { return a.y < 0 || (a.y == 0 && a.x <= 0); } // 精确版
// int half(p2 a) { return a.y < -eps || (fabs(a.y) < eps && a.x < eps); } // eps 版
bool cmp(p2 a, p2 b) { return half(a) == half(b) ? a % b > 0 : half(b); }
bool cmp_eq(p2 a, p2 b) { return half(a) == half(b) && !sgn(a % b); }
int cmp_ct(p2 a, p2 b, p2 c) {
    if (cmp_eq(a, b) || cmp_eq(c, b)) return -1;
    return cmp(a, b) ? cmp(b, c) || cmp(c, a) : cmp(b, c) && cmp(c, a);
}

```

```

sort(ALL(v), cmp);
for (int l = 0, r; l < SZ(v); l = r) {
    for (r = l + 1; r < SZ(v) && cmp_eq(v[l], v[r]); ++r);
    // [l, r) 内向量同向
}

```

```
}  
int order = cmp_ct(a, b, c); // 1: 逆时针, 0: 否, -1: 同向退化
```

## 7.3 seg\_poly

```
// pa 到 pb 的叉积, 逆时针转为正  
db side(p2 p, p2 a, p2 b) { return (a - p) % (b - p); }  
db toL(p2 p, p2 a, p2 b) { return abs(side(p, a, b)) / (b - a).len(); }
```

## 8 Misc

### 8.1 模拟退火

```
// 求最小值; nxt(x, T) 随机生成温度 T 下的邻居。  
auto SA = [&](auto x, double T) {  
    auto bx = x;  
    double v = calc(x), bv = v;  
    for (; T > 1e-9; T *= .995) {  
        auto y = nxt(x, T);  
        double w = calc(y), d = w - v;  
        if (d < 0 || double(rnd()) / rnd.max() < exp(-d / T)) x = y, v = w;  
        if (v < bv) bx = x, bv = v;  
    }  
    return pair{bx, bv};  
};
```

### 8.2 爬山法

```
// 求最小值; near(x, d) 枚举步长 d 下的邻居。  
auto climb = [&](auto x, double d) {  
    double v = calc(x);  
    for (; d > 1e-9; d *= .5) while (1) {  
        auto bx = x;  
        double bv = v;  
        for (auto y : near(x, d)) {  
            double w = calc(y);  
            if (w < bv) bx = y, bv = w;  
        }  
        if (bv == v) break;  
        x = bx, v = bv;  
    }  
    return pair{x, v};  
};
```

8.3 常数表

| $n$   | $\log_{10} n$ | $n!$      | $C(n, n/2)$ | $\text{LCM}(1 \dots n)$ | $P_n$  |
|-------|---------------|-----------|-------------|-------------------------|--------|
| 2     | 0.30102999    | 2         | 2           | 2                       | 2      |
| 3     | 0.47712125    | 6         | 3           | 6                       | 3      |
| 4     | 0.60205999    | 24        | 6           | 12                      | 5      |
| 5     | 0.69897000    | 120       | 10          | 60                      | 7      |
| 6     | 0.77815125    | 720       | 20          | 60                      | 11     |
| 7     | 0.84509804    | 5040      | 35          | 420                     | 15     |
| 8     | 0.90308998    | 40320     | 70          | 840                     | 22     |
| 9     | 0.95424251    | 362880    | 126         | 2520                    | 30     |
| 10    | 1             | 3628800   | 252         | 2520                    | 42     |
| 11    | 1.04139269    | 39916800  | 462         | 27720                   | 56     |
| 12    | 1.07918125    | 479001600 | 924         | 27720                   | 77     |
| 15    | 1.17609126    | 1.31e12   | 6435        | 360360                  | 176    |
| 20    | 1.30103000    | 2.43e18   | 184756      | 232792560               | 627    |
| 25    | 1.39794001    | 1.55e25   | 5200300     | 26771144400             | 1958   |
| 30    | 1.47712125    | 2.65e32   | 155117520   | 1.444e14                | 5604   |
| $P_n$ | 3733840       | 20422650  | 96646760    | 190569292100            | 1e9114 |

| $n \leq$         | 10 | 100 | 1e3 | 1e4  | 1e5  | 1e6   |
|------------------|----|-----|-----|------|------|-------|
| $\max \omega(n)$ | 2  | 3   | 4   | 5    | 6    | 7     |
| $\max d(n)$      | 4  | 12  | 32  | 64   | 128  | 240   |
| $\pi(n)$         | 4  | 25  | 168 | 1229 | 9592 | 78498 |

| $n \leq$         | 1e7    | 1e8     | 1e9    | 1e10   | 1e11   | 1e12   |
|------------------|--------|---------|--------|--------|--------|--------|
| $\max \omega(n)$ | 8      | 8       | 9      | 10     | 10     | 11     |
| $\max d(n)$      | 448    | 768     | 1344   | 2304   | 4032   | 6720   |
| $\pi(n)$         | 664579 | 5761455 | 5.08e7 | 4.55e8 | 4.12e9 | 3.7e10 |

| $n \leq$         | 1e13  | 1e14  | 1e15  | 1e16  | 1e17  | 1e18   |
|------------------|-------|-------|-------|-------|-------|--------|
| $\max \omega(n)$ | 12    | 12    | 13    | 13    | 14    | 15     |
| $\max d(n)$      | 10752 | 17280 | 26880 | 41472 | 64512 | 103680 |

$\pi(n)$ : 质数定理  $\pi(x) \sim x / \log x$ .